

Notebook

February 8, 2026

1 Оракул Блейхенбахера по паддингу

Современные варианты этой атаки были относительно недавно обнаружены [The ROBOT Attack](#), но и оригинальная уязвимость всё ещё скрывается в некоторых старых библиотеках (кстати, похожая уязвимость была исправлена в [OpenSSL 3.0.8](#) 7 февраля 2023).

1.1 Базовый RSA

У примитива RSA ($c = Enc(m) = m^e \bmod N$; $m = Dec(c) = c^d \bmod N$) есть определенные недостатки: 1. Если $m_1 = m_2$, то $c_1 = c_2$, что является проблемой, так как позволяет получить информацию при атаке “человек посередине” (Man-in-the-Middle). (Например, мы можем узнать, что тот же самый человек использует систему в данный момент времени, поскольку он послал тот же самый зашифрованный пароль) 2. Если $e \cdot bitlen(m) < bitlen(N)$, то m^e не переполняет модуль и может быть легко расшифрован взятием e -того корня C .

1.2 PKCS#1 v1.5

Чтобы противодействовать этим проблемам, RSA (компания) создала спецификацию, которая стала [PKCS#1 v1.5](#) (Public Key Cryptography Standards) Спецификация вводит форматирование для шифруемых (или подписываемых) блоков (по-сути, специальный паддинг). Пусть длина модуля N в октетах (байтах) равна k : $k = ||N||$. Данные, которые мы хотим зашифровать или подписать - это D . Тогда шифруемый блок будет выглядеть следующим образом:

$EB = 00 || BT || PS || 00 || D$

где BT принимает значения 00 или 01 для операций на закрытом ключе (создание подписи) и 02 для операций на открытом ключе (шифрование). Шифруемый блок преобразуется в целое число с использованием формата big-endian (первый октет EB - наиболее значимый) PS состоит из $k-3-||D||$ октетов. Их содержимое зависит от BT : 1. $BT = 00$, все октеты в PS принимают значение 00 2. $BT = 01$, все октеты в PS принимают значение FF 3. $BT = 02$, все октеты в PS сгенерированы псевдослучайным образом и не равны 00

Рекомендуется использовать $BT = 01$ для подписи $BT = 02$ для шифрования, потому что 1. При $BT = 01$ с D может быть снято форматирование вне зависимости от его содержимого, а при $BT = 00$ первый байт D , если $D_0 = 00$, тоже

будет снят, что порождает проблемы. 2. И $BT = 01$ и $BT = 02$ генерируют большие числа для шифрования и подписи, поэтому все атаки, зависящие от малого D не работают.

Вы могли заметить, что первый октет EB - 00. Он выбран таким образом, чтобы целое число, полученное из EB всегда было меньше N .

1.3 Атака Блейхенбахера на Оракул по Паддингу

Лучшее всего описал ситуацию д.н. Мэтью Грин в [What is the Random Oracle Model and why should you care? \(Part 4\)](#) (вольный перевод) : >Так, я писал, что ни у кого с практическим складом ума не было претензий к PKCS. Одним ярким исключением был талантливый криптограф из Bell Labs по имени Дэниэль Блейхенбахер. > >У д.н. Блейхенбахера были серьезные претензии к стандарту паддинга PKCS. Вообще-то, можно даже сказать, что он поставил себе цель всей жизни их уничтожить. Эта вендетта достигла апогея в 2006м году, когда он показал, как взламывать популярные имплементации подписи PKCS при помощи листка бумаги и карандаша, незадолго до того как въехал в штаб-квартиру RSA Data Security на минивэне, забитом фейерверками. > >(Так и быть, последней части на самом деле не было, но всё остальное - правда.) > >Блейхенбахер нанёс первый удар по PKCS на конференции CRYPTO 1998. В на удивление читаемой статье он предложил первую практическую адаптивную атаку с выбранным закрытым текстом на "протоколы, использующие стандарт шифрования PKCS #1". Так как, как я уже упоминал, значимым протоколом, который попадал под это определение, был SSL, это было действительно важное событие."

Вы можете взять оригинальную статью здесь: [Chosen Ciphertext Attacks Against Protocols Based on the RSA Encryption Standard PKCS#1](#).

Представим, что мы послали случайное целое число c на расшифровку. Какова вероятность, что снятие паддинга не приведет к ошибке? Первые 2 октета EB должны быть 00||02, а среди остальных октетов должен присутствовать хотя бы один 00. Если k равно 256, то $P = \frac{1}{256^2} \cdot (1 - (\frac{255}{256})^{254}) \approx \frac{1}{104031}$. Кажется, что вероятность слишком маленькая, но на самом деле, она применима на практике. Атака работает следующим образом:

Выбираем c , для которого мы хотим получить открытый текст $m = c^d \bmod N$

Для удобства определим значение $B = 2^{8(k-2)}$, $k = ||N||$. Мы говорим, что шифротекст соответствует PKCS, если он расшифровывается в соответствующий PKCS открытый текст.

Мы выбираем целые числа s , вычисляем шифротексты $c' = cs^e \bmod N$ и отправляем их оракулу. Если c' проходит проверку на ошибку, то $2B \leq ms \bmod N < 3B$. Собрав достаточно различных s мы можем вычислить m . Обычно требуется примерно 2^{20} шифротекстов, но это число сильно варьируется. Атаку можно разделить на три этапа: 1. Маскирование (блайндинг) шифротекста, вычисление c_0 , который соответствует неизвестному m_0 . 2. Поиск таких малых s_i , что $m_0 s_i \bmod N$ соответствует PKCS. На каждый такой s_i атакующий вычисляет интервалы, которые должны содержать m_0 , используя ранее известную информацию. 3. Начинается, когда остается только один интервал. У атакующего достаточно

информации о m_0 , чтобы выбрать s_i , такое что $m_0 s_i \bmod N$ будет соответствовать PKCS с большей вероятностью, чем выбранное случайно сообщение. Постепенно увеличивая s_i , сужается возможный интервал m_0 , пока не останется единственно возможное значение.

(Вообще-то, когда интервал достаточно мал, просто перебрать возможные значения локально более эффективно, чем отправлять запросы оракулу)

1.4 Полный алгоритм (из статьи [paper](#))

M_i - это множество (закрытых) интервалов, которое вычисляется после нахождения успешного (прошедшего проверку) s_i . m_0 содержится в одном из интервалов M_i ### Шаг 1: Блайнддинг. Для целого числа c , выберите различные случайные целые числа s_0 ; после проверьте, обращаясь к оракулу, расшифровываются ли числа $c(s_0)^e \bmod N$ в соответствующие PKCS m' . Для первого успешного s_0 вычисляем:

$$c_0 \leftarrow c(s_0)^e \bmod N$$

$$M_0 \leftarrow \{[2B, 3B - 1]\}$$

$$i \leftarrow 1.$$

Шаг 2: Поиск сообщений, соответствующих PKCS #### Шаг 2.a: Начало поиска. Если $i = 1$, то ищите наименьшее положительное целое $s_1 \geq N/(3B)$, такое что шифртекст $c_0(s_1)^e \bmod N$ соответствует PKCS. #### Шаг 2.b: Поиск при нескольких интервалах. В противном случае если $i > 1$ и количество интервалов в M_{i-1} как минимум 2, то ищите наименьшее целое $s_i > s_{i-1}$, такое что шифртекст $c_0(s_i)^e \bmod N$ соответствует PKCS. #### Шаг 2.c: Поиск при одном интервале. Если M_{i-1} содержит лишь один интервал (например, $M_{i-1} = \{[a, b]\}$), то перебирайте малые целые значения r_i, s_i , такие что

$$r_i \geq 2 \frac{bs_{i-1} - 2B}{N} \tag{1}$$

и

$$\frac{2B + r_i N}{b} \leq s_i < \frac{3B + r_i N}{a}, \tag{2}$$

пока шифртекст $c_0(s_i)^e \bmod N$ не будет соответствовать PKCS.

1.4.1 Шаг 3: Сужение множества решений.

После того, как s_i было найдено, множество M_i может быть вычислено как

$$M_i \leftarrow \bigcup_{(a,b,r)} \left\{ \left[\max \left(a, \left\lceil \frac{2B + rN}{s_i} \right\rceil \right), \min \left(b, \left\lfloor \frac{3B - 1 + rN}{s_i} \right\rfloor \right) \right] \right\} \tag{3}$$

для всех $[a, b] \in M_{i-1}$ и

$$\frac{as_i - 3B + 1}{N} \leq r \leq \frac{bs_i - 2B}{N}$$

Шаг 4: Вычисление решения. Если M_i содержит лишь один интервал длины 1 (например, $M_i = \{[a, a]\}$), то присвойте $m \leftarrow a(s_0)^{-1} \bmod N$, и верните m как решение $m = c^d \bmod N$. В противном случае выставите $i \leftarrow i + 1$ и перейдите на шаг 2.

1.4.2 Замечания.

Шаг 1 может быть пропущен, если c уже соответствует PKCS (например, когда c - зашифрованное сообщение). В этом случае, присвойте $s_0 \leftarrow 1$. Однако шаг 1 всегда необходим для вычисления подписи, даже если Вы не хотите получить “слепую” подпись.

На шаге 2.а, надо начинать с $s_1 = \lceil N/(3B) \rceil$, потому что для меньших значений $m_0 s_1$ никогда не будет соответствовать PKCS.

Мы используем условие (1), потому что за каждую итерацию хотим разделить оставшийся интервал примерно пополам. Также можно использовать перебор на последних стадиях атаки, когда пространство сообщений достаточно маленькое.

1.5 Задание

Вам предоставлен сервер с оракулом Блейхенбахера по паддингу. Реализуйте атаку и дешифруйте шифртекст (он соответствует PKCS). В первом случае вам предоставлена локальная версия сервера, чтобы ускорить тестирование и разработку. Попробуйте использовать функцию **get_pkcs_conforming_mult**, она ускоряет взаимодействие с сервером, что будет особо важно, когда вы будете посылать реальные запросы. Ей просто нужно предоставить список значений

```
[ ]: from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_v1_5
from Crypto.Util.number import bytes_to_long, long_to_bytes
from Crypto.Random import get_random_bytes
class LocalVulnServerClient:
    def __init__(self, show=True):
        """Initialization, generate key"""
        self.key=RSA.generate(2048)
        self.cipher=PKCS1_v1_5.new(self.key)
        self.flag_message="FLAG FOR TESTING"
        self.sentinel=get_random_bytes(16)
    def get_public_key(self):
        return (self.key.e, self.key.n)
    def get_ciphertext(self):
        return bytes_to_long(self.cipher.encrypt(self.flag_message.
↵ encode()))
    def get_pkcs_conforming(self, ciphertext, show=False):
        if isinstance(ciphertext, int):
            ciphertext=long_to_bytes(ciphertext)
        if isinstance(ciphertext, bytes):
            if len(ciphertext)>256:
                print ('Ciphertext too long')
                return None
            if len(ciphertext)<256:
                ciphertext=bytes([0]*(256-len(ciphertext)))+ciphertext
            result=self.cipher.decrypt(ciphertext, self.sentinel)
```

```

        return result!=self.sentinel
    def get_pkcs_conforming_mult(self, ciphertexts, show=False):
        """Check several ciphertexts for PKCS conformity"""
        results=[]
        for ciphertext in ciphertexts:
            if isinstance(ciphertext,int):
                ciphertext=long_to_bytes(ciphertext)
            if isinstance(ciphertext,bytes):
                if len(ciphertext)>256:
                    print ('Ciphertext too long')
                    return None
                if len (ciphertext)<256:
                    ciphertext=bytes([0]*(256-
len(ciphertext)))+ciphertext

                result=self.cipher.decrypt(ciphertext,self.sentinel)
                results.append(result!=self.sentinel)
        return results

```

```

[ ]: vs=LocalVulnServerClient()
(e,N)=vs.get_public_key()
c=vs.get_ciphertext()
print(c)
print (vs.get_pkcs_conforming(c,False))
print (vs.get_pkcs_conforming(2))

# Your code goes here

```

Теперь, когда вы сломали локальный сервер, нужно просто сделать то же самое для удаленного

```

[ ]: import socket
import re
from Crypto.Util.number import inverse, long_to_bytes, bytes_to_long
class VulnServerClient:
    def __init__(self, show=True):
        """Инициализация, подключаемся к серверу"""
        self.s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone', 1344))
        if show:
            print (self.recv_until().decode())
    def recv_until(self, symb=b'\n>'):
        """Получаем сообщения с сервера, по умолчанию до первого,
↪приглашения"""
        data=b''
        while True:

```

```

        data+=self.s.recv(1)
        if data[-len(symb):]==symb:
            break
    return data
def get_public_key(self, show=True):
    """Получаем открытый ключ с сервера"""
    self.s.sendall('public\n'.encode())
    response=self.recv_until().decode()
    if show:
        print (response)
    e=int(re.search('(?<=e: )\d+', response).group(0))
    N=int(re.search('(?<=N: )\d+', response).group(0))
    return (e,N)

def get_ciphertext(self, show=True):
    """Получаем шифротекст с сервера"""
    self.s.sendall('ciphertext\n'.encode())
    response=self.recv_until().decode()
    if show:
        print (response)
    c=bytes_to_long(bytes.fromhex(re.search('(?<=ciphertext: )[0-9a-f]+' , response).group(0)))
    return c

def get_pkcs_conforming(self, ciphertext, show=True):
    """Проверяем шифротекст на соответствие PKCS"""
    if isinstance(ciphertext,int):
        ciphertext=long_to_bytes(ciphertext)
    if len(ciphertext)>256:
        print ('Ciphertext too long')
        return None
    if len (ciphertext)<256:
        ciphertext=bytes([0]*(256-len(ciphertext)))+ciphertext
    ciphertext_hex=ciphertext.hex()

    self.s.sendall(f'unpad {ciphertext_hex}\n'.encode())

    response=self.recv_until().decode()
    if show:
        print (response)
    return response.find('error')== -1

def get_pkcs_conforming_mult(self, ciphertexts, show=False):
    """Проверяем несколько шифротекстов на соответствие PKCS"""
    newc=[]
    for x in ciphertexts:
        if isinstance(x,int):

```

```

        ciphertext=long_to_bytes(x)
    else:
        ciphertext=x
    if len(ciphertext)>256:
        print ('Ciphertext too long')
        return None
    if len (ciphertext)<256:
        ciphertext=bytes([0]*(256-len(ciphertext)))+ciphertext
    ciphertext_hex=ciphertext.hex()
    newc.append(ciphertext_hex)
    self.s.sendall(('\\n'.join([f'unpad {chex}' for chex in_
↪newc]))+'\\n').encode())

    results=[]
    for i in range(len(newc)):
        response=self.recv_until().decode()
        if show:
            print (response)
        results.append(response.find('error')== -1)
    return results

def __del__(self):
    self.s.close()

```

```

[ ]: vs=VulnServerClient()
    (e,N)=vs.get_public_key()
    c=vs.get_ciphertext()
    print (vs.get_pkcs_conforming(c, False))

```

```

[ ]: # Your code goes here

```